SIMATS ENGINEERING

Department of Artificial Intelligence

ITA05 – Computer Vision

ASSIGNMENT 2 – CO2

Apply, Compare and Justify Image Enhancement and Segmentation Techniques for a Smart Document Scanner for the College Library

CO2: Apply image enhancement and segmentation techniques for extracting meaningful regions from images.

STUDENT DETAILS

Particulars	**Details**	**Particulars**	**Details**
Student Name	M. Sahasra	Register Number	192425317
Department / Section	Artificial Intelligece & Machine Learning	Academic Year / Semester	2025-2026
Faculty Name	Dr. Bindu G	Nature of Assignment	Individual
Duration	1 Week	Total Marks	50
Submission Date	_____	Course Code	ITA05

Assignment Title: Apply, Compare and Justify Image Enhancement and Segmentation Techniques for a Smart Document Scanner for the College Library (Total Marks – 50)

MARK SHEET (Rubric-based Evaluation)

Task	Task Title	Max Marks	Marks Awarded	Remarks
1	Grey level transformations and point processing	4		
2	Histogram processing and enhancement	4		
3	Spatial filtering and image smoothing	5		
4	Image sharpening and edge enhancement	5		
5	Frequency-domain low-pass filtering	5		
6	Frequency-domain high-pass filtering	4		
7	Point, line and edge detection	5		
8	Image thresholding and binarization	4		
9	Region-based segmentation and integrated pipeline design	6		

10	Smart scanner adaptation for high-resolution phone photographs	8		
TOTAL		50		

OVERALL REMARKS

Student Signature	Faculty Signature	Program Director

MANDATORY SUBMISSION EVIDENCE CHECK

✓ = present ✗ = missing / not satisfied P = partly present. Based on the "Evidence Expected" of each task. Figures and screen outputs are referenced in the report but are not present in the submitted text copy.

Task	Evidence expected	Status	Notes
1	Transformation curves, before/after pages, histograms and bit-plane images		
2	Histogram/CDF plots, contrast and entropy table, selected CLAHE parameters		
3	Filtered pages, PSNR/SSIM/time table and text intensity profiles		
4	Sharpened pages, stroke profiles and sharpness/noise measurements		
5	Frequency spectra, filtered outputs, ringing measurements and OCR comparison		
6	High-pass outputs, contrast/evenness		

	table and shadow-removal comparison		
7	Detection maps and precision/recall for points, lines and page borders		
8	Binary pages, annotated histograms and Dice scores on at least 10 pages		
9	Stain IoU, corner error, integrated pipeline diagram and stage timings		
10	Three adaptation strategies, blue-note preservation, corner error and 1-second timing		

IMPLEMENTATION EVIDENCE CHAIN (Tasks 1–10)

Stage	Evidence to retain	Used in tasks
Image enhancement	Original pages, transformation outputs, histograms and CDFs	1, 2, 6, 9, 10
Denoising and sharpening	Filtered pages, kernels, profiles, PSNR/SSIM and sharpness	3, 4, 9, 10
Frequency processing	DFT spectra, filter masks, cut-off values and inverse outputs	5, 6, 9, 10
Feature detection	Point, line and edge maps with annotated ground truth	7, 9, 10
Binarization	Binary masks, threshold values and ground-truth comparisons	8, 9, 10
Region segmentation	Stain/punch-hole masks, IoU and leakage examples	9, 10
Page geometry	Detected corners, Hough lines, perspective transforms and crops	9, 10
OCR and validation	Verified text, CER/WER, Dice/IoU and runtime logs	1–10
Decision and adaptation	Design decision table, revised pipeline and timing evidence	9, 10

SUBMISSION CHECKLIST (not separately marked)

- ☐ All ten tasks are answered in the given order.
- ☐ Each task compares all three specified techniques.
- ☐ Every numerical result is labelled as measured, calculated, estimated or illustrative.
- ☐ Python/OpenCV source code is commented and reproducible.
- ☐ Tables include units and clearly stated assumptions.
- ☐ Figures and output screenshots are numbered and captioned.
- ☐ Simulated results are not described as real-world measurements.
- ☐ The final decision in each task is supported by comparison evidence.
- ☐ Original image evidence is retained for audit and verification.

COMPUTER VISION

ASSIGNMENT 2 – CO2

CO2: Apply image enhancement and segmentation techniques for extracting meaningful regions from images.

Nature of Assignment: Individual

Duration: 1 Week

Total Marks: 50

Submission Date: _____

ASSIGNMENT 2 – CO2: IMAGE ENHANCEMENT AND SEGMENTATION

Assignment Title: Apply, Compare and Justify Image Enhancement and Segmentation Techniques for a Smart Document Scanner for the College Library (Total Marks – 50).

Scenario – Smart Document Scanner for the College Library

The library digitizes old printed question papers and lecture notes. Pages may be yellowed, faded, stained by tea, affected by ink blots, photographed under uneven lighting, or contain scanner stripes and dust. The scanner must enhance text, separate text from paper, detect page borders, ruled/table lines, dust specks, punch holes, stains and ink blots, and support cropping and staff review.

Operating Data Used for All Tasks

Parameter	Given value / requirement
Input pages	A4 scans and phone photographs; 8-bit grayscale, 2480 × 3508 pixels at 300 dpi; text strokes 2–4 pixels
Contrast	Most pixels 90–180; faded ink 30–50 grey levels darker than yellowed paper
Lighting	Phone photos brighter at centre with a shadow on one side

Noise	Camera grain σ ≈ 6; about 1% salt-and-pepper dust; faint horizontal scanner stripes every 6 rows
Required outputs	Text, page border, ruled/table lines, dust specks, punch holes, tea stains and ink blots
Targets	Text-paper Dice ≥ 0.90; processing time ≤ 3 seconds/page
Hardware	4 CPU cores, 16 GB RAM, no GPU; Python, OpenCV and NumPy
Dataset	At least 10 own pages plus a public ground-truth dataset such as DIBCO

Experimental Setup and Reporting Method

Use the same image set for all candidate methods. Keep separate ground-truth binary masks and annotations for text, stains, lines and page corners. Record image dimensions, parameter values, elapsed time and software versions. Report actual measurements only. Values in this document are formulas, recommended starting parameters or qualitative expectations; Dice, PSNR, SSIM, OCR error and runtime must be measured by executing the code on the collected dataset.

Shared Processing Workflow

```
import cv2, numpy as np, time
img = cv2.imread("page.png", cv2.IMREAD_GRAYSCALE)
assert img is not None
t0 = time.perf_counter()
# Run the selected enhancement / segmentation stages.
elapsed = time.perf_counter() - t0
print("shape:", img.shape, "dtype:", img.dtype)
print("processing time (s):", elapsed)
```

Dice = $2|A \cap B|/(|A|+|B|)$; IoU = $|A \cap B|/|A \cup B|$. Precision = TP/(TP+FP), recall = TP/(TP+FN). PSNR = $10 \log_{10}(255^2/\text{MSE})$. Use identical masks, tolerances and evaluation conventions for all techniques.

TASK 1 – Basic Grey Level Transformations: Compare Point-Processing Techniques

4 Marks

Enhancement in the spatial domain starts with point operations s = T(r), where each output pixel depends only on the same input pixel. Compare three grey level transformations for making faded text darker and clearer.

Technique	Basic idea and key OpenCV functions
A. Negative + log transformation	s = L – 1 – r (white text on black) and s = c·log(1 + r) to expand dark grey levels; applied with cv2.LUT.
B. Power-law (gamma) transformation	s = c·r^γ; γ > 1 darkens mid-tones (faded ink), γ < 1 brightens them; applied with cv2.LUT.
C. Piecewise-linear contrast stretching	Stretch the 90–180 band to 0–255 using two control points (r1, s1), (r2, s2); optionally grey-level slicing to highlight the ink band.

- Implement three intensity transformation techniques on the same set of page images using Python/OpenCV. Submit commented code, ensuring that all reported results are obtained through code execution or direct measurements.
- Compare the techniques in terms of text-to-paper contrast, detail loss due to clipping, enhancement of shadowed regions, sensitivity to parameters (c, γ, r1, r2), and processing time per page.
- Plot each transformation curve alongside the corresponding input and output image histograms. Calculate the difference between the mean ink intensity and mean paper intensity across 10 pages, and support the conclusions with measured results.

- Perform bit-plane slicing on one page and identify the bit planes that preserve text information and those dominated by noise. Support the observations with the generated bit-plane images.
- Determine the consequences of applying the same (\gamma) value to very light and very dark pages. Examine how contrast stretching affects faint pencil marks and identify the conditions that may cause their loss through clipping.
- Compare one pretrained document-enhancement model with the selected intensity transformation on five faded pages. Measure processing time per page and assess text readability, preservation of faint strokes, shadow removal, and the occurrence of artificially introduced or altered strokes. Report the model, dataset, evaluation procedure, and measured results.

Evidence Expected: Transformation curves, before/after pages and histograms, a contrast table for the three techniques, and a justified choice with parameters.

Ans:

Three techniques

Negative transformation s=255−r reverses black and white and is not suitable as the final reading image, though it can help inspect dark marks. Log transform s=c log(1+r) expands dark values but compresses bright paper values. Gamma transform s=255(r/255)^γ: γ>1 darkens mid-tones; γ<1 brightens. Piecewise-linear stretching maps selected input ranges to wider output ranges and can emphasize the ink band.

Implementation

Use cv2.LUT with a 256-entry uint8 lookup table. For stretching, choose r1 and r2 from the page histogram or measured paper/ink samples; map r1→0 and r2→255 with clipping outside the interval. Do not assume a single mapping suits every page.

Comparison and choice

Negative is mainly an inspection aid. Log improves darker grey levels but may also raise dark noise. Gamma is simple and fast but one γ can over-darken a light page or under-correct a dark page. Piecewise stretching gives direct control over faded ink and paper ranges, so it is the preferred starting point when its control points are calibrated per page.

Bit-plane slicing

For an 8-bit image, bit plane k is (img>>k)&1. Higher planes (typically 6–7) preserve broad paper/text structure; lower planes (0–2) are more affected by grain, dust and compression. Inspect all eight planes rather than assuming fixed plane behaviour.

Parameter and clipping effects

A fixed γ changes different input distributions differently. Very light pages may lose subtle pencil marks after darkening; very dark pages may crush shadows and merge strokes. Stretching clips values below r1 or above r2, so faint pencil marks outside the selected interval may disappear. Use robust percentiles and preserve an unmodified original.

Pretrained model comparison

Compare a documented document-enhancement model on the same five faded pages. Report model name/version, training domain, inference hardware, mean seconds/page, OCR readability and whether faint strokes are removed or hallucinated. A learned model may remove complex degradation better, but may alter real handwriting; retain the classical transform if it is more faithful and meets the time limit.

```
# Three point transformations (8-bit grayscale)
x = img.astype(np.float32) / 255.0
gamma = 1.25
gamma_img = np.uint8(np.clip((x ** gamma) * 255, 0, 255))
log_img = cv2.normalize(np.log1p(img.astype(np.float32)),
                        None, 0, 255, cv2.NORM_MINMAX).astype(np.uint8)
neg_img = 255 - img
```

TASK 2 – Histogram Processing: Compare Histogram-Based Enhancement 4 Marks

Pages from different years and cameras have very different brightness, so their histograms differ. Compare three histogram processing techniques.

Technique	Basic idea and key OpenCV functions
A. Global histogram equalization	Map grey levels through the cumulative distribution function (CDF) of the whole page; cv2.equalizeHist.
B. Histogram matching (specification)	Make each page's histogram match that of one clean reference page; implement the CDF-to-CDF mapping yourself.
C. Local (adaptive) histogram equalization	Equalize small tiles separately with a clip limit; cv2.createCLAHE(clipLimit, tileGridSize).

- Execute all three selected enhancement techniques using Python/OpenCV on the same set of page images. Submit commented code and ensure that every reported result is based on actual execution or direct measurement.
- Characterize the techniques in terms of text clarity, noise amplification in blank paper regions, shadow correction, visibility of tile boundaries, consistency across light and dark pages, and processing time per page.
- Generate input and output histograms and cumulative distribution functions (CDFs) for three representative pages: light, normally illuminated, and shadowed. Calculate and

report image contrast and entropy for each technique, supported by plots and numerical results.

- Examine local mean and variance statistics to establish why global histogram equalization may fail to correct spatially varying shadows. Justify the selected CLAHE tile size and clip limit using experimental results.
- Determine the effects of applying global histogram equalization to a page containing approximately 90% blank paper. Assess the consequences of using an excessively high CLAHE clip limit on a noisy mobile-phone photograph, including noise amplification and loss of text readability.

Evidence Expected: Histogram/CDF plots, before/after pages, a contrast/entropy table for the three techniques, and justified parameter choices.

Ans:

Global equalization

cv2.equalizeHist uses the cumulative distribution function of the whole page to spread grey levels. It is fast and may improve overall contrast, but a large blank-paper area dominates the histogram and spatial shadows remain because all locations use one mapping.

Histogram matching

Calculate the source and reference CDFs. For each source intensity, map it to the reference intensity with the closest CDF value. This makes pages more consistent with a selected clean reference, but the reference must be representative; an unsuitable reference can make faded text too light or amplify stains.

CLAHE

cv2.createCLAHE(clipLimit, tileGridSize) equalizes local tiles while limiting histogram amplification. It is best suited to uneven lighting because each local area is enhanced separately. Excessive clipLimit amplifies grain, dust and scanner stripes; very small tiles can create visible tile boundaries.

Evaluation

For light, normal and shadowed pages, plot input/output histograms and CDFs. Measure contrast (for example, paper mean minus ink mean) and entropy $H=-\Sigma p(i)\log_2 p(i)$. Also measure blank-region standard deviation to detect noise amplification. Compare all methods on identical regions.

Recommended setting

Use CLAHE as the leading candidate for phone images with spatially varying shadows. Start with tileGridSize around (8,8) and a modest clipLimit (for example 2.0), then tune on the validation set. For uniformly lit scans, global equalization or a calibrated reference match may be faster and sufficient.

90% blank page and excessive clipping

When about 90% of pixels are blank paper, global equalization allocates most of its mapping to paper tones; the small text population may receive limited separation and noise can become more visible. A very high CLAHE clip limit boosts local noise and stains, making the background mottled and reducing OCR reliability.

```
clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))
clahe_img = clahe.apply(img)
equalized = cv2.equalizeHist(img)
```

TASK 3 – Spatial Filtering and Image Smoothing: Compare Smoothing Filters 5 Marks

Camera grain and dust specks must be removed without making the thin (2–4 pixel) text strokes blurry or broken. Compare three smoothing spatial filters.

Technique	Basic idea and key OpenCV functions
A. Box (averaging) filter	Linear filter: average of the m × m neighbourhood; cv2.blur.
B. Gaussian filter	Linear filter with a Gaussian-weighted mask; cv2.GaussianBlur(img, (k, k), σ).
C. Median filter	Non-linear filter: median of the neighbourhood; cv2.medianBlur.

- Execute three selected image-denoising techniques using Python/OpenCV on the same set of page images. Submit commented code and ensure that all reported results are obtained through code execution or direct measurements.
- Characterize the techniques in terms of grain removal versus dust-speck removal, text-stroke sharpness, preservation of small characters such as “i”, “.” and “,”, sensitivity to kernel sizes of 3 × 3, 5 × 5 and 7 × 7, and processing time per page.
- Introduce known noise into clean page images and calculate PSNR and SSIM for each denoising technique. Plot grayscale intensity profiles across a line of text before and after filtering to demonstrate changes in noise, stroke sharpness and character preservation.
- Manually calculate the output of a 3 × 3 filter using both correlation and convolution. Verify the results against cv2.filter2D and justify the selection of

cv2.BORDER_REFLECT or zero padding based on boundary artefacts and text preservation.

- Determine the consequences of using a 7 × 7 box filter to remove dust specks and omitting smoothing before subsequent edge-detection and thresholding operations. Support the discussion with relevant image outputs.
- Assess a classical denoising filter against a pretrained model, such as DnCNN or a diffusion based restoration model, on the same noisy page images. Evaluate restoration quality, processing time per page, preservation of small text characters, and the risk of hallucinated or altered strokes.

Evidence Expected: Filtered pages, a PSNR/SSIM/time table, text profiles, and a justified filter and kernel size.

Ans:

Box filter

cv2.blur averages all pixels in the neighbourhood. It reduces random grain but blurs edges uniformly and can erase punctuation and thin strokes, especially with 5×5 or 7×7 kernels.

Gaussian filter

cv2.GaussianBlur weights nearby pixels more strongly. It smooths camera grain with less abrupt artefact than a box filter, but still softens fine text. Choose a small kernel such as 3×3 and suitable σ.

Median filter

cv2.medianBlur replaces each pixel with the neighbourhood median. It is effective for isolated salt-and-pepper specks while preserving edges better than averaging. A 3×3 median is the preferred first test; larger kernels may remove dots and punctuation.

PSNR and SSIM

Add known Gaussian noise and salt-and-pepper noise to clean images. Compare restored images to the clean original using PSNR and SSIM. Also inspect text crops and intensity profiles across letters; a high PSNR alone does not guarantee preservation of punctuation.

Correlation and convolution

Correlation slides a kernel without flipping it. Convolution flips the kernel horizontally and vertically before sliding. Symmetric box/Gaussian/median neighbourhoods produce equivalent results under kernel reversal; an asymmetric derivative kernel changes sign/orientation. Verify a small matrix manually against cv2.filter2D.

Border handling and kernel choice

BORDER_REFLECT mirrors pixels at the boundary and usually avoids artificial dark frames. Zero padding inserts black values and can create false edges around page boundaries. A 7×7 box filter averages across 2–4 pixel strokes and can erase punctuation; no smoothing allows grain and dust to trigger many false edges and threshold components.

Deep model comparison

Compare a named DnCNN/restoration model with 3×3 median or Gaussian on the same noisy pages. Record PSNR, SSIM, OCR CER/WER, seconds/page and altered-stroke examples. On a CPU-only PC, classical filters are usually more practical; learned restoration is acceptable only if it preserves exact text and meets runtime.

```
box = cv2.blur(img, (3, 3))
gauss = cv2.GaussianBlur(img, (3, 3), 0.8)
median = cv2.medianBlur(img, 3)
```

TASK 4 – Spatial Filtering and Image Sharpening: Compare Sharpening Filters 5 Marks

Phone photos are slightly out of focus, so letters look soft. Compare three sharpening filters based on first and second derivatives.

Technique	Basic idea and key OpenCV functions
A. Laplacian sharpening	Second derivative: $g = f - c \cdot \nabla^2 f$ using a 4- or 8-neighbour Laplacian mask; cv2.Laplacian.
B. Unsharp masking / high-boost	$g = f + k \cdot (f - \text{blurred } f)$, with $k = 1$ (unsharp) or $k > 1$ (high-boost); cv2.GaussianBlur + cv2.addWeighted.
C. Gradient (first-derivative) sharpening	Gradient magnitude from Sobel masks (cv2.Sobel in x and y) added to the page to strengthen edges.

- Execute three selected image-sharpening techniques using Python/OpenCV on the same set of page images. Submit commented code and ensure that all reported results are obtained through code execution or direct measurements.
- Characterize the techniques in terms of letter sharpness, noise amplification in blank regions, bright halos around letters, effects on stains, and the need for prior smoothing.
- Plot grayscale intensity profiles across representative letter strokes. Calculate a sharpness measure, such as variance of the Laplacian, and a noise measure within blank paper regions for each filter. Support the findings with plots and numerical results.
- Interpret the experimental results to establish why second-derivative filters can emphasize fine intensity transitions more strongly than first-derivative filters. Justify the

processing sequence of smoothing followed by sharpening using observed noise levels and edge profiles.

- Determine the consequences of applying high-boost filtering with a large amplification factor (k) to a noisy, unsmoothed photograph. Examine the effects of displaying a Laplacian output without appropriate scaling when negative pixel values are clipped.
- Assess one pretrained deep deblurring or super-resolution model against unsharp masking on the same set of blurred phone photographs. Evaluate OCR accuracy using character error rate (CER) or word error rate (WER), processing time, preservation of small characters, and the risk of hallucinated or altered text strokes.

Evidence Expected: Sharpened pages, stroke profiles, a sharpness/noise table for the three filters, and a justified filter with parameters.

Ans:

Laplacian

The Laplacian is a second derivative that responds strongly to rapid intensity changes. Sharpen with $g=f-c\nabla^2 f$ for the usual sign convention. It emphasizes fine details but also amplifies noise and may produce halos. Convert the signed response carefully before display.

Unsharp masking

Blur the image, calculate the high-frequency mask f−blur(f), and add it back: $g=f+k(f-\text{blur}(f))$. k=1 is unsharp masking; k>1 is high-boost. It is easy to tune and generally provides controlled text sharpening.

Gradient sharpening

Compute Sobel gx and gy, then magnitude $\text{sqrt}(gx^2+gy^2)$. Add a scaled magnitude to the original. It strengthens edges but can make all edges, including stains and dust, prominent.

Measurements

Plot a grey-level profile across representative letter strokes. Compute variance of Laplacian as a sharpness indicator and standard deviation in blank paper as a noise indicator. Higher sharpness is useful only if blank-region noise and halos remain controlled.

Why second derivative responds strongly

The second derivative measures changes in the gradient, so it highlights narrow transitions and fine detail. First derivatives measure edge strength and direction. The Laplacian can react strongly to small noise fluctuations; mild smoothing before sharpening reduces false responses.

Failure conditions and selection

Large high-boost k on an unsmoothed photo creates bright/dark halos and amplifies grain. Laplacian outputs contain negative values; directly converting them to uint8 clips negative values and gives misleading results. Use float32, combine with the original, then clip. Prefer mild unsharp masking after light denoising.

Deep deblurring comparison

Compare unsharp masking with a named deblurring/super-resolution model using OCR character error rate (CER) or word error rate (WER), small-character preservation, runtime and visual inspection. Super-resolution can invent plausible but incorrect strokes; never treat generated detail as ground truth.

```
blur = cv2.GaussianBlur(img, (3, 3), 0.8)
unsharp = cv2.addWeighted(img, 1.6, blur, -0.6, 0)
lap = cv2.Laplacian(blur, cv2.CV_32F, ksize=3)
lap_sharp = np.clip(img.astype(np.float32) - 0.5*lap, 0, 255).astype(np.uint8)
```

TASK 5 – Frequency-Domain Enhancement: Compare Low Pass Filters [5 Marks]

Frequency-domain enhancement follows a fixed process: pad the image, centre the transform, compute the DFT F(u, v), multiply by a filter H(u, v), take the inverse DFT, and crop. Compare three low pass filters for removing grain and the scanner stripes.

Technique	Basic idea and key OpenCV functions
A. Ideal low pass filter (ILPF)	H = 1 inside cut-off radius D0 and 0 outside; cv2.dft or np.fft.fft2 with np.fft.fftshift.
B. Butterworth low pass filter (BLPF)	H = 1 / (1 + (D/D0)^(2n)); smooth change controlled by order n.
C. Gaussian low pass filter (GLPF)	$H = \exp(-D^2 / 2D0^2)$; no sharp cut-off.

- Measure the impact of frequency-domain scanner-stripe removal on text recognition using Tesseract 5 or PaddleOCR. Compare OCR performance before and after filtering using verified reference text, character error rate (CER) or word error rate (WER), processing time, and representative recognition errors.
- Substantiate the effectiveness of frequency-domain filtering by presenting the log-magnitude spectrum of a scanned page, identifying frequency components associated with scanner stripes, and reporting filtered outputs and quantitative ringing measurements for each filter at all three (D_0) values.
- Verify the equivalence of frequency-domain Gaussian filtering and spatial-domain Gaussian filtering using equivalent parameters. Quantify the difference between their

outputs and assess the computational conditions under which frequency-domain processing provides a practical time advantage.

- Critique the effects of omitting appropriate padding before the discrete Fourier transform and selecting an Ideal Low-Pass Filter with an abrupt cut-off. Support the conclusions with observed wrap-around artefacts, ringing measurements, and changes in text-edge quality.
- Appraise the effectiveness of frequency-domain scanner-stripe removal for OCR using Tesseract 5 or PaddleOCR. Compare recognition performance before and after filtering using verified reference text, character error rate (CER) or word error rate (WER), processing time, and representative recognition errors. Recommend whether the filtering stage should be retained, modified, or omitted based on the measured evidence.

Evidence Expected: Spectra, filtered pages for three cut-offs, a ringing/blur/time table for the three filters, and a justified filter and D0.

Ans:

Frequency-domain process

Pad the image, compute DFT, shift the zero-frequency component to the centre, multiply by H(u,v), inverse-shift and inverse-transform, then crop to the original size. Use floating-point arrays and retain the real component.

Ideal low pass

H=1 for D≤D0 and 0 otherwise. Its abrupt cut-off can cause ringing (Gibbs phenomenon) near sharp text edges. It can remove high-frequency grain but may blur narrow strokes.

Butterworth low pass

H=1/[1+(D/D0)^(2n)]. Order n controls transition steepness. It offers a compromise between sharp cut-off and smooth attenuation; high n approaches the Ideal filter and can increase ringing.

Gaussian low pass

$H=\exp(-D^2/(2D0^2))$. Its smooth transition reduces ringing, but may blur text if D0 is too small. It is generally the safest low-pass candidate for document images.

Stripe analysis and OCR

Inspect the log-magnitude spectrum. Periodic horizontal stripes create concentrated frequency energy in the direction perpendicular to the stripes. Design a targeted notch/band-reject filter when possible; a general low-pass may remove text detail along with stripe energy. Evaluate Tesseract 5 or PaddleOCR using verified reference text and CER/WER before and after.

Cut-off and padding

Test at least three D0 values (e.g. low, medium, high relative to the image dimensions). Smaller D0 removes more noise but blurs strokes; larger D0 preserves text but leaves noise. Padding reduces circular wrap-around at image borders. Without padding, opposite page edges can interact and create artefacts.

Spatial versus frequency Gaussian

A Gaussian in frequency corresponds to a Gaussian-like smoothing operation in space, subject to discrete sampling, kernel truncation and boundary handling. Compare outputs with mean absolute difference and runtime. For a single page, spatial Gaussian is often simpler; FFT processing becomes useful for large images or repeated filtering with the same transform.

Recommendation

Use Gaussian low-pass only when spectrum inspection shows diffuse high-frequency noise. For regularly spaced scanner stripes, use a narrow notch filter targeted to measured stripe frequencies and verify OCR. Keep the stage only if CER/WER improves without loss of thin strokes.

```
f = np.fft.fftshift(np.fft.fft2(img.astype(np.float32)))
rows, cols = img.shape
u, v = np.ogrid[:rows, :cols]
D = np.sqrt((u-rows/2)**2 + (v-cols/2)**2)
D0 = 45.0
H = np.exp(-(D**2)/(2*D0**2))       # Gaussian low-pass
```

```
out = np.real(np.fft.ifft2(np.fft.ifftshift(f*H)))
out = np.clip(out, 0, 255).astype(np.uint8)
```

TASK 6 – Frequency-Domain Enhancement: Compare High Pass Filters [4 Marks]

High frequencies carry the letter edges; low frequencies carry the slow change in brightness caused by the shadow. Compare three high pass approaches for clear text on a shadowed page.

Technique	Basic idea and key OpenCV functions
A. Ideal / Butterworth high pass filter	$H_{hp} = 1 - H_{lp}$ (ideal and Butterworth forms); output scaled for display.
B. Gaussian high pass with high-frequency emphasis	$H = a + b \cdot H_{hp}$ ($a \geq 0$, $b > a$), so the paper tone is kept while letter edges are strengthened.
C. Homomorphic filtering	Uses the illumination–reflectance model $f = i \cdot r$: take log, apply a high-emphasis filter ($\gamma L < 1$, $\gamma H > 1$), then exp, to remove the shadow (illumination) and keep the text (reflectance).

- Implement three selected frequency-domain enhancement techniques using Python/OpenCV on the same set of shadowed and normally illuminated page images.

Submit commented code and validate all reported results using actual execution outputs and quantitative measurements.

- Assess the techniques in terms of text sharpness, preservation of average image brightness (DC component), ringing artefacts, shadow suppression, and noise amplification.
- Calculate text contrast and the standard deviation of pixel intensities in blank paper regions across each page. Compare the frequency-domain Laplacian results with those obtained using the spatial-domain Laplacian in Task 4, supporting the conclusions with quantitative measurements and representative output images.
- Critique the consequences of applying an unmodified Ideal High-Pass Filter (HPF) output directly to a document digitisation pipeline, considering loss of paper brightness and overall tonal information. Assess the effects of interchanging the low-frequency and high-frequency gain parameters γL and γH in homomorphic filtering.
- Appraise homomorphic filtering against one pretrained deep learning-based shadow-removal model on the same shadowed document photographs. Compare shadow uniformity, text contrast, stroke preservation, noise amplification, processing time, and OCR performance using CER or WER against verified reference text. Recommend whether the classical or learned approach should be retained based on the measured evidence.

Evidence Expected: High-pass outputs, a contrast/evenness table for the three approaches, and a justified choice with parameters.

Ans:

Ideal/Butterworth HPF

$H_{hp}=1-H_{lp}$. High-pass filtering suppresses low frequencies and retains rapid changes such as text edges. A pure HPF also removes the DC component, so the page background becomes dark and tonal information is lost. Butterworth has a smoother transition than Ideal.

Gaussian high-frequency emphasis

Use $H=a+bH_{hp}$, with $a \geq 0$ and $b > a$. The low-frequency gain a retains average page brightness while b strengthens text edges. This is more suitable than a pure HPF for a readable document.

Homomorphic filtering

Model image $f(x,y)=i(x,y)r(x,y)$, where i is illumination and r is reflectance. Take logarithm so multiplication becomes addition, transform to frequency domain, apply low-frequency gain $\gamma_L < 1$ and high-frequency gain $\gamma_H > 1$, inverse transform and exponentiate. This suppresses slow illumination variation while retaining text detail.

Quantitative comparison

Measure text contrast (paper mean minus ink mean), standard deviation in blank paper regions, average output brightness, ringing and OCR CER/WER. Compare frequency-domain Laplacian results with spatial Laplacian results from Task 4 using the same page and scale.

Parameter interpretation

If γ_L and γ_H are interchanged, low-frequency illumination is boosted and high-frequency text detail is attenuated; shadows may become stronger and text weaker. An unmodified HPF output should not be used directly for digitization because paper brightness and broad tonal information are lost.

Deep shadow removal

Compare homomorphic filtering with one pretrained shadow-removal model on the same shadowed pages. Measure shadow uniformity, text contrast, stroke preservation, noise, time and OCR error. Use the classical method if it meets quality and time targets with fewer risks; inspect learned outputs for altered or invented strokes.

```
# Homomorphic filtering: log -> DFT -> emphasis filter -> inverse -> exp
log_img = np.log1p(img.astype(np.float32))
F = np.fft.fftshift(np.fft.fft2(log_img))
# Construct H with low-frequency gain gamma_L and high-frequency gain gamma_H.
# Apply H to F, inverse-transform, exponentiate, normalize for display.
```

TASK 7 – Image Segmentation: Compare Point, Line and Edge Detection [5 Marks]

Dust specks behave like isolated points, ruled and table lines like lines, and the page border like a step edge. Compare three detection approaches.

Technique	Basic idea and key OpenCV functions
A. Point and line detection masks	Laplacian point mask with $\|R\| \geq T$ for dust specks; four 3×3 line masks (0°, 45°, 90°, –45°) for ruled and table lines; cv2.filter2D.
B. Gradient edge operators	Roberts, Prewitt and Sobel masks; gradient magnitude and direction, then a threshold; cv2.Sobel, cv2.magnitude, cv2.phase.
C. Advanced edge detection	Marr–Hildreth (Laplacian of Gaussian with zero crossings) and Canny (smoothing, non-maximum suppression, hysteresis); cv2.Canny.

- Implement three selected point, line, and edge detection techniques using Python/OpenCV on the same set of document images. Submit commented source code and execution outputs for all reported results.
- Judge the techniques based on their ability to detect dust specks, ruled lines, and page borders, considering noise sensitivity, thick or double edges, false detections caused by text and scanner stripes, and processing time.
- Calculate precision and recall against hand-annotated ground-truth images, using a matching tolerance of 2 pixels. Report separate results for dust specks, ruled lines, and

page borders, and use the measurements to identify the most reliable technique for each feature.

- Defend the selected threshold (T) for point detection and both Canny thresholds using experimental results. Determine why the four directional line masks may fail to detect lines when the page is slightly tilted, and recommend suitable modifications.
- Diagnose the effects of applying edge detection directly to noisy document photographs and treating every detected letter or character as a point feature. Support the conclusions with representative output images and false-positive or missed-detection measurements.
- Recommend whether Canny or a pretrained learned edge detector, such as Holistically-Nested Edge Detection (HED), is more suitable for detecting page borders and table lines in cluttered photographs. Use the same test images and ground truth to compare precision, recall, false detections, edge continuity, and processing time, and support the recommendation with measured results.

Evidence Expected: Detection maps, a precision/recall table for points, lines and edges, and a justified detector for each item.

Ans:

Point and line masks

A Laplacian point mask responds to isolated intensity changes; threshold $|R| \geq T$ to locate dust specks. Directional 3×3 masks at 0°, 45°, 90° and −45° detect ruled lines. cv2.filter2D applies the masks. Text punctuation can also look like points, so exclude text regions or use size/shape constraints.

Gradient operators

Roberts, Prewitt and Sobel estimate first derivatives. Compute gradient magnitude and direction, then threshold. Sobel includes some smoothing and is convenient in OpenCV; noise can still create many responses.

Marr–Hildreth and Canny

Marr–Hildreth smooths with Gaussian, applies Laplacian and detects zero crossings. Canny smooths, computes gradients, performs non-maximum suppression and hysteresis thresholding. Canny generally gives thin, connected edges when thresholds are suitable.

Ground-truth evaluation

Hand annotate dust specks, ruled lines and page borders separately. Match detected pixels to ground truth within 2 pixels. Calculate precision and recall for each class, not one combined score. Also report false positives from text and scanner stripes.

Threshold and tilted lines

Select point threshold T from validation precision-recall trade-off. Tune Canny low/high thresholds together; too low creates noise edges and too high misses faint borders. Fixed directional masks may miss tilted rules because their orientation is between the four templates. Use more orientations, rotate/deskew the page, or use Hough line detection.

Failure cases and learned edge model

Running edge detection on noisy photos creates dense false edges; treating every letter as a point incorrectly marks characters as dust. Compare Canny with HED using identical images and labels. HED may give more continuous semantic edges but is heavier and may require GPU; on the stated CPU hardware Canny is the practical default unless HED clearly improves measured border/line quality within the time limit.

```
blur = cv2.GaussianBlur(img, (5, 5), 1.0)
edges = cv2.Canny(blur, 50, 150)
sobelx = cv2.Sobel(blur, cv2.CV_32F, 1, 0, ksize=3)
sobely = cv2.Sobel(blur, cv2.CV_32F, 0, 1, ksize=3)
mag = cv2.magnitude(sobelx, sobely)
```

TASK 8 – Image Segmentation: Compare Thresholding Techniques [4 Marks]

To print the page clearly or send it to OCR, every pixel must become text (black) or paper (white). The shadow makes this hard. Compare three thresholding techniques.

Technique	Basic idea and key OpenCV functions
A. Basic global thresholding	Iterative algorithm: T = average of the two class means, repeated until T stops changing; cv2.threshold.
B. Otsu's optimum global thresholding	Choose T that maximizes the between-class variance; cv2.threshold(..., THRESH_BINARY + THRESH_OTSU).
C. Variable (adaptive) thresholding	Threshold changes across the page (local mean or Gaussian mean); cv2.adaptiveThreshold.

- Develop and execute three image-thresholding techniques using Python/OpenCV on a common dataset of document images. Submit commented source code and execution outputs, ensuring that all reported results are reproducible from the implemented code.
- Assess the performance of the three techniques using Dice similarity against ground-truth masks, text preservation in shadowed regions, black-patch formation over stained areas, character fragmentation or merging, and sensitivity to parameter variations.
- Generate intensity histograms for the test images and mark the selected threshold (T) on each histogram. Tabulate Dice scores for at least 10 document pages, including shadowed and stained samples, and determine which technique provides the most consistent binarization performance.
- Validate the selected global and adaptive threshold parameters using experimental results. Apply the illumination–reflectance model $f = i \cdot r$ to explain how non-uniform

illumination affects global thresholding. Determine whether Gaussian smoothing, as investigated in Task 3, improves Otsu thresholding by examining changes in Dice scores.

- Diagnose the effects of applying Otsu thresholding to shadowed mobile-phone photographs and selecting an adaptive threshold block size smaller than the character dimensions. Quantify text loss, character fragmentation, and false foreground regions, and support the findings with output images.
- Select the more suitable approach between the best-performing classical thresholding technique and a pretrained deep-learning binarization model trained on DIBCO document images. Evaluate both approaches on the same test pages using Dice score, text preservation, stain-related artefacts, character connectivity, and processing time. Substantiate the selection with measured results.

Evidence Expected: Binary pages, annotated histograms, a Dice table for the three techniques, and a justified thresholding method.

Ans:

Basic iterative threshold

Start with an initial T, divide pixels into two classes, calculate class means μ1 and μ2, and update T=(μ1+μ2)/2 until stable. It works when foreground and background are reasonably separable but is sensitive to uneven illumination.

Otsu threshold

Otsu chooses T that maximizes between-class variance. Use cv2.threshold with THRESH_BINARY+THRESH_OTSU. It is automatic and effective for a bimodal histogram, but a page shadow can create multiple background modes and a poor global split.

Adaptive threshold

cv2.adaptiveThreshold computes a local threshold using a neighbourhood mean or Gaussian-weighted mean minus C. It handles spatially varying shadows, but block size must be larger than character structures and suited to local illumination changes.

Evaluation

Compare binary masks with ground truth using Dice on at least 10 pages, including light, shadowed and stained examples. Plot histograms and mark global T. Record text loss, merged/fragmented characters, black stain patches and false foreground.

Illumination model and smoothing

With f=i·r, varying illumination i changes observed pixel values even when reflectance r (text/paper) is unchanged; therefore a single global threshold may fail across shadow and bright regions. Test Gaussian smoothing before Otsu and compare Dice; smoothing can reduce noise but excessive smoothing weakens narrow strokes.

Parameter failures and model comparison

Otsu on a shadowed phone photo can classify the dark shadow as foreground, producing large black patches. Adaptive block size smaller than character dimensions can fragment strokes and create speckled masks. Compare the best classical method with a DIBCO-trained binarization model using Dice, character connectivity, stain artefacts and CPU time. Select based on both fidelity and runtime.

```
_, otsu = cv2.threshold(img, 0, 255,
                        cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU)
adaptive = cv2.adaptiveThreshold(img, 255,
   cv2.ADAPTIVE_THRESH_GAUSSIAN_C, cv2.THRESH_BINARY_INV, 31, 10)
```

TASK 9 – Region-Based Segmentation, Edge Linking and Boundary Detection: Compare and Integrate [6 Marks]

Stains and ink blots must be found as whole regions, and the page border found as one closed boundary so the page can be cropped. Compare three approaches, then join your choices from Tasks 1–8 into one smart-scanner pipeline.

Technique	Basic idea and key OpenCV functions
A. Region growing	Start from a seed pixel inside a stain and add 8-connected neighbours whose grey level is close to the seed; implement yourself or use cv2.floodFill.
B. Region splitting and merging	Split the page into quadrants until each region is uniform (quadtree), then merge neighbouring regions that are similar; implement yourself.
C. Edge linking and boundary detection	Join edge pixels with similar strength and direction, use the Hough transform (cv2.HoughLines / HoughLinesP) to find the four page sides and table lines, and trace boundaries with cv2.findContours.

- Develop and execute three selected document-segmentation techniques using Python/OpenCV on a common dataset of at least 10 document images. Submit commented source code and execution outputs, ensuring that all reported results are obtained from actual code execution.
- Assess the techniques based on stain-region detection completeness, leakage into clean paper or text, closure of gaps in page boundaries, page-corner localization accuracy, parameter sensitivity, and processing time.
- Calculate stain-region Intersection over Union (IoU), page-corner localization error in pixels, and processing time per page for each technique. Present the results in a

comparative table and identify the approach that best satisfies the segmentation accuracy and runtime requirements.

- Construct and Validate Construct a block diagram of the complete processing pipeline, including grayscale transformation or histogram processing, smoothing, sharpening or frequency-domain filtering, point/line/edge detection, thresholding, region growing or edge linking, and final page cropping with stain annotations. Report the execution time of each stage, determine whether the total processing time meets the 3-second limit, and prepare a Design Decision Table containing Decision, Evidence-Based Justification, and Consequences if Not Selected.

- Verify whether the measured results support or contradict the predictions made in Tasks 1–8. Substantiate the conclusions using quantitative results and representative output images, and identify the factors responsible for any discrepancies.

- Diagnose the effects of selecting a seed point on a text character rather than a stain during region growing, and determine how an interrupted page boundary caused by shadows affects edge linking and page cropping. Propose and substantiate suitable corrective measures using experimental results.

Select the most suitable approach among region growing, watershed segmentation, and a promptable foundation model such as Segment Anything Model 2 (SAM 2) for separating stains, punch holes, and page boundaries. Evaluate all approaches on the same test images using stain-region IoU, boundary accuracy, text preservation, processing time, and manual intervention.

Record the number of prompts or corrective interactions required by SAM 2 per page and justify the final selection using measured evidence.

Evidence Expected: Segmentation results and a comparison table for the three approaches, an original block diagram with timings, a decision table, and a defended final pipeline.

Ans:

Region growing

Choose a seed inside a stain and add 8-connected neighbours whose grey level differs from the seed or evolving region mean by less than a tolerance. It can capture irregular stains, but a poor seed on text may spread through connected characters or leak into paper.

Splitting and merging

Quadtree splitting divides non-uniform regions into quadrants; merging combines adjacent regions with similar statistics. It can represent large stains and paper regions but may be computationally expensive and sensitive to the homogeneity threshold.

Edge linking and Hough

Join edge pixels with compatible strength and direction. HoughLines/HoughLinesP estimates straight page sides and table rules; findContours traces closed boundaries. This is effective for page geometry but shadows may interrupt edges, requiring morphological closing and line fitting.

Metrics

For stain masks, calculate IoU. For page corners, calculate Euclidean localization error in pixels against manually labelled corners. Report runtime/page, leakage into text, boundary closure and sensitivity. Use at least 10 pages.

Integrated pipeline

Recommended sequence: grayscale or suitable colour-channel extraction → mild CLAHE/contrast normalization → 3×3 median for impulse dust or mild Gaussian for grain → optional mild unsharp mask → point/line/edge detection → adaptive threshold for text → morphology/contours → Hough page-side estimation → perspective crop → stain/punch-hole annotations.

Pipeline block diagram

Input page → enhancement → denoising → text binarization + point/line/edge maps → region segmentation → page-boundary linking → perspective correction/crop → annotated scan + OCR-ready binary output.

Timing and decisions

Measure every stage separately with time.perf_counter and sum stage times. If total exceeds 3 seconds, resize for detection, use ROI processing, avoid repeated FFTs and choose efficient morphology. A decision table should record selected method, measured evidence, and the cost of not selecting it.

Seed and interrupted boundary failures

A seed on a text character can make region growing absorb connected strokes; select seeds from stain candidates or use multiple validated seeds and region constraints. A shadow-interrupted page edge breaks contours and page cropping; use illumination correction, morphological closing, Hough line extrapolation and quadrilateral validation.

SAM 2 comparison

Compare region growing, watershed and SAM 2 on the same pages. Record stain IoU, boundary accuracy, text preservation, time and manual prompts/corrections per page. SAM 2 can be flexible but prompt dependence and CPU cost may be high. A deterministic classical pipeline is preferred for routine high-volume scanning if it meets accuracy and time limits.

```
edges = cv2.Canny(cv2.GaussianBlur(img, (5,5), 1), 50, 150)
closed = cv2.morphologyEx(edges, cv2.MORPH_CLOSE,
                          np.ones((5,5), np.uint8))
contours, _ = cv2.findContours(closed, cv2.RETR_EXTERNAL,
                               cv2.CHAIN_APPROX_SIMPLE)
# Approximate the largest suitable contour to a four-corner polygon.
```

TASK 10 – Re-apply and Adapt the Smart Scanner Pipeline [8 Marks]

Changed Requirement: The library now lets students upload pages from their own phones. Photos are 12-megapixel colour JPEGs (4000 × 3000 pixels) taken under classroom tube lights, often at an angle, with light blue pen notes in the margins that must be kept. JPEG compression adds small 8 × 8 block patterns. The result must now appear within 1 second per page on the same lab PC.

Technique	Basic idea and key OpenCV functions
A. Retain	Convert the colour photo to grey level and run the Task 9 pipeline unchanged.
B. Modify	Resize to a smaller working size, re-tune CLAHE, filters and adaptive threshold, and choose the colour channel in which the blue pen is darkest.
C. Redesign	Homomorphic filtering for uneven lighting, frequency-domain filtering for JPEG block patterns, Hough-based page detection with perspective correction (cv2.getPerspectiveTransform), then thresholding.

- Develop and execute three document image processing strategies using Python/OpenCV on at least 15 phone photographs captured under classroom lighting. Submit commented source code and execution outputs, ensuring that all reported results are obtained from actual code execution.
- Critique the assumptions underlying the original pipeline regarding image resolution, grayscale processing, illumination, page orientation, noise characteristics, and execution

time. Determine which assumptions are invalid for the new dataset and assess the consequences of applying the Task 9 pipeline without modification.

- Audit the performance of the original transformations, filters, detectors, and thresholding methods on the new photographs. Measure the execution time per page at a resolution of (4000 \times 3000) pixels, identify the stages that remain effective, and determine which require modification or replacement to meet the 1-second limit.
- Rank Strategies A, B, and C using measured Dice score, preservation of blue handwritten annotations, page-corner localization error in pixels, and processing time per page. Select the best-performing strategy and formulate a revised pipeline, identifying the modifications required to improve accuracy and efficiency.
- Substantiate the selected strategy using quantitative measurements, Task 9 results, compliance with the 1-second processing limit, and documented performance trade-offs. Identify one operating condition under which the selected strategy would no longer be suitable and justify this limitation using experimental evidence.
- Determine which stages of the revised pipeline could be replaced or assisted by a document-AI model such as LayoutLMv3 or Donut. Assess the roles of conventional preprocessing, geometric correction, text recognition, and layout interpretation, and justify which computer-vision stages must remain to satisfy the required output specifications.

Evidence Expected: Three adaptation strategies applied and compared with measured evidence, and a RETAIN / MODIFY / REDESIGN decision that follows logically from the changed constraints.

Ans:

Changed assumptions

The original grayscale 2480×3508 input, fixed orientation, scanner-like noise, and 3-second budget no longer apply. Colour is essential because blue annotations must be preserved;

perspective distortion and JPEG blocking are new problems. Processing 12 MP at full resolution on four CPU cores is substantially more expensive.

Strategy A – Retain

Convert to grayscale and run the Task 9 pipeline unchanged. This is simplest but risks losing blue pen notes, cannot correct perspective robustly, and may exceed 1 second. It is a baseline, not the preferred design.

Strategy B – Modify

Resize to a working resolution for detection, tune CLAHE and denoising, and choose a colour channel where blue ink is darkest (often the red channel, but verify from sample RGB values). Preserve the original colour image and map detected geometry back to full resolution. This is the recommended starting strategy.

Strategy C – Redesign

Use homomorphic illumination correction, targeted frequency filtering for JPEG blocks, Hough-based page detection, perspective transform and then thresholding. It addresses more degradation sources but adds computation and may remove real blue marks if applied to grayscale indiscriminately.

Blue annotation preservation

Compare B, G and R channels and HSV/Lab representations on annotated pages. Blue ink often has low response in the red channel relative to paper, but actual pen and lighting vary. Create a blue-ink mask in HSV/Lab and combine it with the text mask so margin notes are retained in the final output.

JPEG artefacts and perspective

JPEG blocking appears near 8×8 block boundaries; mild deblocking or carefully selected low-pass filtering may help, but strong smoothing damages 2–4 pixel text. Detect the page quadrilateral on a reduced image, order its four corners, compute cv2.getPerspectiveTransform and warpPerspective.

Revised pipeline and runtime

Recommended: load colour image → resize a copy for fast page detection → detect four corners and estimate transform → perspective-correct original image → retain blue-ink mask → mild colour/illumination normalization → light denoising → adaptive threshold for printed text → merge preserved blue annotations → crop/export. Measure each stage and optimize until the total is ≤1 second/page.

Ranking and evidence

Rank A, B and C using Dice, blue-note preservation, corner error and seconds/page. Do not invent results: execute on at least 15 photos and report measured values. A strategy that meets the time target but erases blue notes is not acceptable.

Limitation

The modified strategy may fail when the page is heavily folded, partly occluded, very dark, or has no reliable four-corner boundary. In such cases manual corner correction or a more advanced document detector may be required.

Document-AI role

LayoutLMv3 or Donut can assist with document understanding, layout and text extraction after geometric correction. They do not replace camera geometry correction, pixel-level stain masks, page-border detection or preservation of blue annotations. Keep conventional CV preprocessing and geometric correction for these required visual outputs; use document-AI for semantic interpretation/OCR where appropriate.

```
# Three point transformations (8-bit grayscale)
x = img.astype(np.float32) / 255.0
gamma = 1.25
gamma_img = np.uint8(np.clip((x ** gamma) * 255, 0, 255))
log_img = cv2.normalize(np.log1p(img.astype(np.float32)),
                        None, 0, 255, cv2.NORM_MINMAX).astype(np.uint8)
neg_img = 255 - img
```

Overall Conclusion and Final Evidence Check

For the original library scanner, the preferred design is a restrained enhancement pipeline: local contrast correction for uneven illumination, small-kernel denoising, carefully controlled sharpening, adaptive thresholding for text, and geometry/region methods for page cropping and stain annotations. Frequency-domain filtering should be included only when the spectrum shows a specific periodic or illumination problem and OCR measurements demonstrate benefit.

For the changed 12 MP phone-photo requirement, MODIFY is the most practical initial decision: perform page detection on a reduced copy, correct perspective on the original colour image, preserve blue annotations using colour information, and apply only lightweight enhancement and binarization. REDESIGN is justified only if measured results show that the additional frequency-domain and homomorphic stages improve quality while still meeting the one-second limit.

Evidence item	**What to include in final practical submission**
Dataset	At least 10 original pages plus public ground-truth pages; at least 15 phone photos for Task 10
Implementation	Commented Python/OpenCV source code, environment and parameter settings
Visual evidence	Before/after images, histograms/CDFs, bit planes, spectra, profiles, binary masks, detection maps and cropped pages
Quantitative evidence	Dice, IoU, precision, recall, PSNR, SSIM, CER/WER, corner error and per-stage/per-page runtime
Comparison	Same test images and evaluation protocol for every candidate method
Decision	A reasoned technique selection, limitations, and a timing check against 3 seconds/page or 1 second/page as applicable

Marks Distribution

Task	Marks
Task 1 – Grey Level Transformation Analysis	4
Task 2 – Histogram Processing and Enhancement	4
Task 3 – Spatial Filtering and Image Smoothing	5
Task 4 – Image Sharpening and Edge Enhancement	5
Task 5 – Frequency-Domain Low-Pass Filtering	5
Task 6 – Frequency-Domain High-Pass Filtering	4
Task 7 – Point, Line, and Edge Detection	5
Task 8 – Image Thresholding and Binarization	4
Task 9 – Region-Based Segmentation and Integrated Pipeline Design	6
Task 10 – Smart Scanner Adaptation for High-Resolution Phone Photographs	8
TOTAL	50